# Technical Build Plan (100% Free / Open-Source)

### Multi-Agent Generative AI System for Time-Series Forecasting & Anomaly Detection

### Astronomy (light curves) + Financial Markets (price + sentiment)

**Owner:** Kaveri Neeli (solo BTP) · **Horizon:** 3 months · **Start:** 2026-07-26 · **Target finish:** 2026-10-25 **Dev hardware:** CPU-only laptop, with optional lab parallel-computing system later for benchmarking **Constraint:** No paid APIs during development. Paid cloud LLMs (OpenAI/Claude) are reserved for the final demo only (see `PROFESSOR_PROPOSAL.md`).

---

## 0. Answering the two questions you asked directly

**"Can I build this using free APIs?"** — Yes, entirely. Nothing in this architecture requires a paid key until the very last week (optional) and the live demo.

**"Do I have to build my own embeddings?"** — Not from scratch in the sense of training a BERT-sized language model (no time/compute for that in 3 months, and it wouldn't be a good use of BTP time). Instead, "own embeddings" here means two legitimate, free things:

1. **Text embeddings:** run a small *pretrained, open-source* embedding model **locally** (e.g. `sentence-transformers/all-MiniLM-L6-v2` or `BAAI/bge-small-en-v1.5`, both free, Apache/MIT licensed, ~100–130MB, CPU-fast). This avoids the OpenAI/Cohere embeddings API entirely — zero cost, zero API key.
2. **Light-curve (time-series) embeddings:** you genuinely train your own small encoder (autoencoder / 1D-CNN) on light-curve windows to produce a latent vector per curve. This one really is "built by you" and is a nice, defensible thesis contribution, since no off-the-shelf pretrained embedding model for astronomical flux time-series is standard the way MiniLM is for text.

So: pretrained-and-free for text, self-trained-and-small for time-series. Both are free-API-compliant.

---

## 1. System Architecture

```
flowchart TB
    U[User: pick ticker OR light-curve target] --> R[Orchestrator / Router Agent]
    R -->|finance query| FA[Finance Agent]
    R -->|astronomy query| AA[Astronomy Agent]

    FA --> FD[(yfinance OHLCV)]
    FA --> SA[Sentiment Agent]
    SA --> NEWS[(RSS/News + Reddit, free)]
    SA --> FinBERT[FinBERT local sentiment model]
    FA --> PRICEMODEL[ARIMA baseline + Gradient-Boosted model]

    AA --> LC[(Kepler/TESS light curves via lightkurve)]
    AA --> DETECT[Isolation Forest / Autoencoder anomaly detector]
    AA --> TSEMB[Custom time-series encoder -> embedding]

    FA --> EXP[Explainer Agent]
    AA --> EXP
    TSEMB --> VDB[(FAISS/Chroma vector store)]
    SA --> VDB
    VDB --> EXP
    EXP --> LLM[Local LLM via Ollama\n(llama3.2:3b / qwen2.5:3b)]
    LLM --> OUT[Final natural-language report + charts]
```

**Agents (5, all coordinated in one LangGraph state graph):**

| Agent | Responsibility | Tools it calls |
|---|---|---|
| Orchestrator/Router | Classifies incoming request as astronomy or finance (or both), manages shared state | none (routing logic + tiny LLM call) |
| Astronomy Agent | Runs anomaly detection pipeline on a light curve, produces structured findings | `lightkurve`, Isolation Forest/Autoencoder, custom encoder |
| Finance Agent | Runs price forecast pipeline, pulls technical indicators | `yfinance`, ARIMA (existing repo code), XGBoost/LightGBM |
| Sentiment Agent | Fetches news/social text, scores sentiment, embeds text | RSS/PRAW, FinBERT, MiniLM embeddings |
| Explainer/Report Agent | Retrieves similar past cases from vector store, synthesizes everything into a human-readable report | FAISS/Chroma retrieval, Ollama LLM |

---

## 2. Tech Stack (everything free)

| Layer | Choice | Why |
|---|---|---|
| Agent orchestration | **LangGraph** (open-source) | Explicit state graph → doubles as your architecture diagram for the report. CrewAI is an easier fallback if LangGraph's learning curve eats too much time. |
| Local LLM runtime | **Ollama** | Free, runs quantized GGUF models on CPU. Models: `llama3.2:3b`, `qwen2.5:3b`, `phi3:mini`, fallback `gemma2:2b` if too slow. |
| Text embeddings | `sentence-transformers` (`all-MiniLM-L6-v2` or `bge-small-en-v1.5`) | Free, local, ~100ms/sentence on CPU. |
| Custom time-series embeddings | Small PyTorch/Keras 1D-CNN or LSTM autoencoder | You train this yourself on light-curve windows — this is the "own embeddings" deliverable. |
| Vector store | **FAISS** (faiss-cpu) or **Chroma** | Both free, local, no server needed. |
| Astronomy data | **lightkurve** + NASA MAST archive (public), or Kaggle "Kepler Labelled Time Series Data" for a faster bootstrap | Free, public domain. |
| Astronomy anomaly models | Isolation Forest / One-Class SVM (scikit-learn) baseline, small autoencoder for reconstruction-error anomaly scoring | CPU-feasible, no GPU needed at this scale. |
| Financial data | **yfinance** (already in your existing repo) | Free, no key. |
| News/sentiment data | Yahoo Finance RSS + Google News RSS (no key needed), Reddit via **PRAW** (free developer app) | Avoids paid/rate-limited news APIs entirely. |
| Sentiment model | **FinBERT** (`ProsusAI/finbert`, HuggingFace, free) | Purpose-built for financial text, runs on CPU. |
| Price model | ARIMA (existing `model_train.py` baseline) + **XGBoost/LightGBM** with technical + sentiment features | Gradient-boosted trees train in seconds on CPU and are a fair, legitimate comparison point — better ROI than a deep LSTM on a laptop. |
| Front-end | Streamlit (you already know it from the stock app) | Free, fast to wire up for a demo. |

**Nothing above requires a credit card.** The only paid thing anywhere in this plan is the optional final-demo LLM swap to OpenAI/Claude, which is a config change, not a rewrite (see §6).

---

## 3. Astronomy Track — Anomaly Detection

1. **Data:** Pull light curves for 100–300 Kepler/TESS targets via `lightkurve` (or start faster with the Kaggle Kepler labelled CSV dataset — has known exoplanet-transit labels you can use as *proxy ground truth* for evaluating anomaly detection, since a transit dip is a legitimate anomaly).
2. **Preprocessing:** normalize flux, remove long-term trends (detrending), handle NaNs/gaps, window into fixed-length segments.
3. **Baseline detector:** extract statistical features per window (mean, std, skew, kurtosis, BLS period estimate) → Isolation Forest / One-Class SVM.
4. **Learned detector:** small 1D-CNN or LSTM autoencoder trained on "normal" windows only; reconstruction error = anomaly score. This same encoder's bottleneck layer doubles as your **custom light-curve embedding**.
5. **Evaluation:** precision/recall/ROC-AUC against the labelled transit dataset; visualize flagged anomalies vs missed/true transits.

## 4. Financial Track — Prediction + Sentiment

1. **Price data:** extend your existing `pages/utils/model_train.py` — you already have ARIMA + RMSE evaluation working. Keep it as the baseline.
2. **Sentiment pipeline:** scrape headlines (RSS, no key) + optionally Reddit (PRAW, free) for a basket of ~10–20 tickers → run FinBERT locally → daily sentiment score per ticker.
3. **Augmented model:** XGBoost/LightGBM using technical indicators (already partly built — RSI/MACD in `Stock_Analysis.py`) + rolling sentiment score as features, forecasting next-day/next-30-day direction or price.
4. **Evaluation:** RMSE/MAE with vs without sentiment features — this comparison is your key financial-track result.

## 5. Embeddings + RAG Layer

- Embed: (a) news/headline text via MiniLM, (b) light-curve windows via your custom autoencoder, (c) past agent-generated reports.
- Store all three in one FAISS/Chroma index (namespaced by type).
- Explainer Agent retrieves top-k similar past cases (e.g. "this anomaly looks like the transit-like dip flagged in target X") to ground the LLM's final summary — this is what makes the system "RAG-augmented" rather than just an LLM hallucinating commentary.

## 6. Cloud-API Swap-In (build this abstraction early, use it late)

From Week 7 onward, wrap your LLM call behind one function, e.g. `call_llm(prompt, backend="ollama")`, so that swapping `backend="openai"` or `backend="claude"` for the final demo is a one-line config change,

not a rewrite. Keep this abstraction in mind now even though you won't flip it on until the demo — retrofitting it later costs more than building it in from the start.

## 7. Local-vs-Lab Benchmark (turns your professor's suggestion into a real experiment)

Run identical workloads — same Ollama model, same batch of embeddings, same autoencoder training run — on your laptop CPU vs the lab's parallel-computing system once it's available. Measure: tokens/sec, end-to-end agent-pipeline latency, autoencoder training time per epoch. This becomes a "System Feasibility" section in your report and directly answers the email's open question about which setup is suitable.

---

## 8. Week-by-Week Schedule (2026-07-26 → 2026-10-25)

### Month 1 — Domain MVPs (no agents yet)

| Week | Dates | Goal | Deliverable |
|---|---|---|---|
| 1 | Jul 27–Aug 2 | Env setup: Python env, Ollama + pull a 3B model, HF transformers, sentence-transformers, lightkurve, faiss-cpu. Skim 3–5 papers per domain + 1 on multi-agent LLM frameworks. Init this repo. | Repo scaffolded, reproducible env, 1-page lit summary |
| 2 | Aug 3–Aug 9 | Data pipelines: pull light curves (lightkurve or Kaggle Kepler set); extend yfinance loader to a ticker basket; stand up RSS/PRAW scraping | Two clean versioned datasets + loaders + EDA notebook |
| 3 | Aug 10–Aug 16 | Astronomy anomaly detector v1: features + Isolation Forest baseline, evaluate vs transit labels | Working detector + eval report (precision/recall/AUC) |
| 4 | Aug 17–Aug 23 | Financial predictor v1: FinBERT sentiment pipeline + XGBoost model with sentiment features, compare vs ARIMA-only | Working sentiment-augmented predictor + comparison table |

**Checkpoint (~Aug 23):** both domains work standalone as scripts. Demo to professor.

### Month 2 — Embeddings, RAG, Multi-Agent Assembly

| Week | Dates | Goal | Deliverable |
|---|---|---|---|
| 5 | Aug 24–Aug 30 | Custom embeddings: MiniLM for text; train small autoencoder for light-curve latent vectors | Two embedding pipelines, saved vectors |
| 6 | Aug 31–Sep 6 | Vector store: FAISS/Chroma index across text + time-series + reports; build top-k retrieval | Working local RAG retrieval demo |
| 7 | Sep 7–Sep 13 | Pick LangGraph; build one working agent end-to-end (detector → retrieval → local LLM summary) | One agent working fully, LLM-backend abstraction in place |
| 8 | Sep 14–Sep 20 | Build out full agent graph: Orchestrator, Astronomy, Finance, Sentiment, Explainer as LangGraph nodes | Multi-agent graph runs end-to-end for both domains |
| 9 | Sep 21–Sep 27 | Wire into Streamlit front-end: input → agent trace → final report | Usable end-to-end demo app (open-source stack) |

**Checkpoint (~Sep 27):** full multi-agent pipeline works locally, entirely free stack. Demo to professor.

## Month 3 — Benchmarking, Cloud Path, Evaluation, Writing

| Week | Dates | Goal | Deliverable |
|---|---|---|---|
| 10 | Sep 28–Oct 4 | Local-vs-lab benchmark (or CPU-only proxy if lab access slips) | Benchmark table/plots for report |
| 11 | Oct 5–Oct 11 | Flip the LLM-backend abstraction to OpenAI/Claude for testing (small budget/trial credits only — full paid use is reserved for the live demo) | Working cloud-backed version, latency comparison |
| 12 | Oct 12–Oct 18 | Full evaluation + ablations (astro metrics, finance RMSE with/without sentiment, embedding visualization via t-SNE/UMAP) | Results section draft, all report figures |
| 13 | Oct 19–Oct 25 | Write report, build slides, rehearse live demo on cloud-API path | Final report draft, slides, rehearsed demo |

---

## 9. Suggested Repo Structure

```
BTP-multiagent-timeseries/
├── astronomy/
│   ├── data_loader.py        # lightkurve / Kaggle ingestion
│   ├── anomaly_detector.py   # Isolation Forest + autoencoder
│   └── ts_embedder.py        # custom light-curve encoder
├── finance/
│   ├── price_data.py         # yfinance loader (reuse from stock repo)
│   ├── sentiment.py          # RSS/PRAW scraping + FinBERT
│   └── price_model.py        # ARIMA baseline + XGBoost augmented model
├── agents/
│   ├── llm_backend.py        # call_llm(prompt, backend="ollama"|"openai"|"claude")
│   ├── graph.py               # LangGraph state graph definition
│   └── nodes/                 # one file per agent node
├── rag/
│   ├── embed_text.py          # MiniLM wrapper
│   └── vector_store.py        # FAISS/Chroma wrapper
├── app.py                     # Streamlit front-end
├── benchmarks/                # local-vs-lab timing scripts + results
├── requirements.txt
└── README.md
```

## 10. Risks & Mitigations

| Risk | Mitigation |
|---|---|
| CPU-only laptop too slow for LLM agent loop | Use smallest capable Ollama model (3B or less); cache LLM calls during dev; only run full pipeline end-to-end occasionally, iterate on sub-pieces standalone |
| Light-curve data volume too large for laptop | Subsample targets (100–300, not the full archive); work on windows, not full curves |
| News/RSS scraping brittle or blocked | Have 2 fallback sources (Yahoo RSS + Google News RSS + Reddit); cache scraped data locally so pipeline doesn't depend on live scraping during dev |
| LangGraph learning curve eats Week 7 | Fallback to CrewAI (simpler role-based API) if not productive within ~3 days |
| Lab parallel-computing system not ready by Week 10 | Benchmark section can run CPU-only vs a larger local quantized model as a proxy; note the gap honestly in the report |